MAPLESURE INSURANCE · AMS CONSOLE

S3 Enhancement Delivery

Test & demo guide — all scenarios

Generated 27 July 2026 · repo `ams-s3-demo` · branch `deploy-bedrock-aws`
Companion docs: `docs/design/S3_DESIGN.md` (technical design) · `demo/DEMO_TEST_GUIDE.md` (presenter script)

Environment

Five services make up the full demo environment. All five must be running before you start.

PORT	SERVICE	URL	START COMMAND (TERMINAL BY TERMINAL)
8000	FastAPI backend the engine — never on screen	`localhost:8000`	`uvicorn apps.console.api.main:app --port 8000`
5173	**AMS Console** — start here login, then `/s3`	`localhost:5173`	`cd frontend && npm run dev`
8501	MapleSure portal (CR-041 / CR-042 target) "the client's app"	`localhost:8501`	`demo/run_mockapp.sh`
8081	ClaimsPortal Policy Team console (CR-043)	`localhost:8081`	`apps/run-policy-service.sh`
8082	ClaimsPortal Claims Team console (CR-043)	`localhost:8082`	`apps/run-claims-service.sh`

Both ClaimsPortal services together, in one foreground terminal: `demo/run_s3_springdemo.sh`. It traps `kill 0` on exit, so use the two separate commands above if you want them in independent background terminals.

Direct URLs for showing raw data

URL	SHOWS
`localhost:8000/docs`	OpenAPI browser for the whole S3 API
`localhost:8081/api/policies`	All ClaimsPortal policies as JSON
`localhost:8081/api/policies/MS-1004`	One policy — the clearest place to show `deductible` appearing
`localhost:8082/api/claims`	Submitted claims with decision and `payableAmount`

> **THREE THINGS THAT WILL BITE YOU**
>
> **Never run uvicorn with `--reload`.** The pipeline writes `.py` files into the tree uvicorn watches; the reload wipes the in-memory session store and every request then 401s while the UI still looks logged in.
>
> **The ClaimsPortal processes serve whatever was last running.** There's no build step (plain Python), but `uvicorn` is deliberately started without `--reload`, so an Apply changing the source does not take effect until you restart both services. A process already running survives the reset entirely: it keeps serving the *post-CR* app while the source sits at baseline, and both ports answer 200 the whole time. Port-is-up is not process-is-current — check the process start time against your reset time.
>
> **Start the portal only via `demo/run_mockapp.sh`.** The script exports `PYTHONPATH="$PWD"` first. Streamlit otherwise puts `apps/policycore/` on `sys.path` instead of the repo root, and the app dies at `ModuleNotFoundError: No module named 'common'`. This one hides: a portal already running from a bad launch keeps serving its last good render until something re-executes the script — and a reset rewriting `app.py` is exactly that, so the error surfaces *during the reset* and looks like the reset broke it. It didn't.

Logins

Fictional roster; passcode is `1001 + roster position`.

NAME	PASSCODE	ROLE IN THESE TESTS
Ravi Kumar	1001	Developer — owns AMS-102, AMS-103
Elena Cruz	1002	Second engineer — owns AMS-098
Priya Nair	1003	Tester — owns AMS-101; QA hand-off target
Tom Becker	1004	Second tester
Manager	9000	Manager view

Seeded board

TICKET	CR	STATUS	ASSIGNEE	TARGET	LANGUAGE
AMS-101	CR-2026-041	QA	Priya Nair	mockapp — coverage tier	Python
AMS-102	CR-2026-042	In Progress	Ravi Kumar	mockapp — endorsement priority	Python
AMS-103	CR-2026-043	To Do	Ravi Kumar	spring-demo — claims deductible	Python
AMS-098	—	Done	Elena Cruz	none (ad-hoc path)	—

Reset between runs

The pipeline mutates real files. Stop the services first, then run all three in **this order** — the endorsement reset must come *after* `reset_s3.sh`, because it restores the superset mockapp state that leaves both AMS-101 and AMS-102 demo-ready.

```
demo/reset_s3.sh              # CR-041 + shared state: .cache/llm, out/, events, Jira caches
demo/reset_s3_endorsement.sh  # CR-042 baseline  (MUST run after reset_s3.sh)
demo/reset_s3_springdemo.sh   # CR-043 target, from apps/claimsportal/.baseline/
```

Then restart all five services. Hard-refresh the browser tab — the console caches per-ticket state in `localStorage` and drops it when the reset marker changes.

> **EXPECTED AFTER A RESET**
>
> `git status` shows `apps/policycore/{app,core/db,core/endorsements,core/models}.py` as modified. That is **correct** — repo HEAD is committed in a post-CR-2026-042 state, so the pre-CR baseline necessarily differs from it.
>
> Resets also clear `.cache/llm`, so impact analysis, design doc and release notes make live provider calls again (~10–20 s each on first click). Codegen and testgen always replay — instant and free. Run `demo/warm_s3_cache.sh` if you want the narrative beats instant too.

Before → after: what changes on screen

The "show the client their own app changing" moment for each scenario. Have this open while presenting.

SCENARIO	WHERE TO LOOK	BEFORE — BASELINE	AFTER APPLY	TO SEE IT
AMS-101 CR-2026-041	Portal :8501 Policy Detail	No coverage tier anywhere in the app.	Tier per policy (Standard / Premium / *the name the audience picked*), upgrade control on the detail view, premium recalculates and persists.	Browser refresh
AMS-102 CR-2026-042	Portal :8501 Request a Policy Endorsement	Exactly 5 fields — endorsement type, description, effective date, contact phone, contact email.	A 6th field, **Priority** (Standard / Urgent, defaults Standard). Existing submit flow unchanged.	Browser refresh
AMS-103 CR-2026-043	Claims :8082 and Policy :8081	Policies carry no deductible. An **$80 claim on MS-1004 → ACCEPTED**.	Policies gain a deductible (MS-1001 500, MS-1002 1000, MS-1003 500, MS-1004 100). The same **$80 claim on MS-1004 → REJECTED_BELOW_DEDUCTIBLE**. A **$1,200 claim on MS-1001 → ACCEPTED, `payableAmount` 700**.	**Restart both services**

> **THE EASIEST WAY TO MAKE A WORKING DEMO LOOK BROKEN**
>
> The two mockapp scenarios need only a browser refresh — Streamlit re-executes `app.py` on each run. **Scenario 3 does not.** The running processes serve whatever was loaded at startup (no `--reload`), so a just-applied CR is invisible on :8081/:8082 until you restart. Apply the change, switch to :8082, and nothing has changed. Ctrl-C both terminals, then `demo/run_s3_springdemo.sh`.

Confirm the baseline is genuinely "before"

Run after the resets, before you present. Every line must match the comment.

```
grep -c 'coverage_tier\|COVERAGE_TIERS' apps/policycore/app.py    # 0       — no CR-041 yet
grep -c 'selectbox("Priority"' apps/policycore/app.py              # 0       — no CR-042 yet
ls apps/policycore/core/coverage.py                                # absent
ls apps/claimsportal/claims_service/claim_rules.py          # absent
ls apps/policycore/core/endorsements.py                            # PRESENT — CR-042's "before" needs it
```

```
curl -s localhost:8081/api/policies/MS-1004                # no "deductible" key
ls s3_enhancement/out/ | wc -l                             # 0
```

Then confirm all five services are up *and fresh* — every start time must be later than your last reset:

```
for p in 8000 5173 8501 8081 8082; do
  pid=$(lsof -ti tcp:$p -sTCP:LISTEN | head -1)
  code=$(curl -s -o /dev/null -w '%{http_code}' --max-time 5 http://localhost:$p/)
  printf "%-5s http=%-3s pid=%-6s %s\n" "$p" "$code" "${pid:-DOWN}" \
    "$(ps -o lstart= -p ${pid:-0} 2>/dev/null)"
done
```

Use `-sTCP:LISTEN`. Without it a Chrome tab connected to :5173 matches before the real listener and you read the wrong process's start time.

Scenarios

01 AMS-103 — ClaimsPortal, full pipeline

Start as: Ravi Kumar (1001) → hand off to Priya Nair (1003) · **Time:** ~15 min · **Best single end-to-end run**

1. Open :5173 → *Open application* → click **AMS-103**.
2. **Run AI impact analysis.** It asks up to two clarifying questions — a gap question, then an assumption question. Answer each in the same box.
3. Close the modal → **Stage 1 · Generate the change**.
4. **Apply to repo.**
5. **Stage 2 · Draft design doc** → **Assign tester & move to QA** (Priya Nair).
6. Log out → log in as **Priya Nair** → open AMS-103 → **Stage 3 · Generate tests** → **Run tests** (real pytest, same runner as the other two scenarios).
7. **Prove the tests catch bugs** → **Stage 4 · Draft release notes** → mark Done.

- ☐ After the second answer it *must* produce a final analysis — the two-turn cap is hard.
- ☐ Activity tab logs: clarification requested ×2 → impact analysis drafted → status In Progress.
- ☐ Selection panel reads **Files in this app 5 · used 5** and *"No subsystem doc matched closely enough"* — no `apps/policycore/systems/…` entry.
- ☐ Token panel says *"no saving over whole-app context here"* — the honest branch.
- ☐ Six Python files, each with a one-line reason.
- ☐ Post-apply link reads **"open the Claims Team console"** → :8082 (not the mockapp portal).
- ☐ On disk: `claim_rules.py` created, `deductible` present in `policy.py`.
- ☐ Stage 3 was *locked* for Ravi — the developer cannot verify their own change.
- ☐ Per-case checklist parsed from pytest's JUnit XML output, all green.
- ☐ Mutation flips `<=` to `<`; the "claim at exactly the deductible" test goes red; file auto-restored (`git diff` on `claim_rules.py` is clean afterwards).

> **:8081 WILL NOT CHANGE — BY DESIGN**
>
> CR-2026-043 line 50 states *"Both team consoles must keep rendering without changes to their HTML"*, and no `.html` is in the target's codegen allowlist. The policy console's table has no Deductible column, so it looks identical after the CR. The change *is* there — `GET :8081/api/policies` now returns `"deductible":500`.
>
> **The visible proof is on :8082.** Restart the services, then submit a claim against MS-1001 (limit 25 000, deductible 500):

CLAIM AMOUNT	BEFORE THE CR	AFTER THE CR
400	ACCEPTED	REJECTED_BELOW_DEDUCTIBLE
500 — exactly the deductible	ACCEPTED	REJECTED_BELOW_DEDUCTIBLE
1 200	ACCEPTED	ACCEPTED — payable 700 now recorded
99 000	REJECTED_OVER_LIMIT	REJECTED_OVER_LIMIT

02 AMS-101 — the scoring showcase

> **As:** Priya Nair (1003) — the ticket is seeded to her, already in QA, so she runs the whole flow · **Time:** ~10 min

1. Click **AMS-101** → **Run AI impact analysis**.
2. **Stage 1 · Generate** — type an audience-chosen tier name (≤ 24 chars, no quotes or slashes, e.g. *Platinum*).
3. **Apply** → post-apply runs `python -m apps.policycore.core.seed`.
4. **Design doc** → **Generate tests** → **Run** (pytest) → **Mutation check** → **Release notes**.

- ☐ Panel reads **Files in this app 56 · used 4**.
- ☐ Expand "6 other subsystems screened out" → bar chart, struck through: settlement 0.33, underwriting 0.29, audit 0.29, risk 0.28, billing 0.20, reporting 0.10 — all under the 0.45 floor. **50 Java decoy files never opened.**
- ☐ Token panel shows a real multiplier against whole-app context.
- ☐ Your chosen tier name appears verbatim in the generated `coverage.py` — one recording serves any name.
- ☐ Refresh **:8501** → the portal now has the coverage-upgrade control.
- ☐ Mutation weakens the same-tier guard `<=` → `<`; the suite goes red, then reverts.

03 AMS-102 — endorsement priority

> **As:** Ravi Kumar (1001) → hand off to a tester · Same step sequence as Scenario 01

> **DO NOT CLICK APPLY IN FRONT OF AN AUDIENCE**
>
> The committed CR-042 codegen recording is a **net-deletion recording**: across four files it removes 81 lines and adds 12, stripping roughly six function docstrings from `apps/policycore/core/db.py` plus many blank lines. The diff a reviewer sees is dominated by deletions unrelated to the CR.
>
> Viewing the diff to *demonstrate* the problem is fine. The fix is a live re-record against `s3-endorsement-baseline`. The CR-041 and CR-043 recordings are clean.

- ☐ Diff shows only three files with pending changes — the fourth returned byte-identical and was dropped.
- ☐ If applied: **:8501** endorsement form gains a *Priority* dropdown (Standard / Urgent, Standard default).

04 The review loop

Where: any ticket, Stage 1, after Generate and before Apply

ACTION	EXPECTED
Show diff on a file	Per-file unified diff, collapsible
Ask about this file → *"why did you remove the docstring?"*	Answers **from its own diff** — admits the removal instead of denying it
Ask → *"rename that variable to X"*	Returns a revised file; still staged, still not applied
Ask a pure question	Answers in prose and changes **no** code
Add to proposal → `apps/policycore/core/claims.py` + an instruction	File joins the same reviewed diff
Apply a single file	Others stay pending; re-applying is a no-op

- ☐ Throughout: `git status` stays clean until Apply. Nothing reaches the repo before then.

05 Cross-team impact & the dependency gate

Needs two logins · Ravi Kumar (1001) + whoever the new ticket is assigned to

1. As Ravi, open AMS-102 or AMS-103 → **Check for other teams affected**.
2. Confirm a suggested team → a linked Jira ticket is created and assigned.
3. Log in as that assignee → open their ticket → **Mark as Done**.
4. Log back in as Ravi.

- ☐ After step 2, Stage 1 is blocked: *"Waiting on 1 other team to finish their ticket."*
- ☐ After step 3, the gate clears for Ravi — state comes from the append-only events log, not the browser, which is why it crosses separate logins.
- ☐ No ticket is ever created without a human confirming it.

06 Ad-hoc ticket & the clarity gate

As: Elena Cruz (1002) · **Ticket:** AMS-098 — no linked CR, so it routes to the ad-hoc path

- ☐ Runs with **no codebase context** — correctly shows no file-selection panel.
- ☐ Three gates share one two-question budget: overall vagueness → a specific missing detail → repo identity.
- ☐ A repo match below *high* confidence asks *"is this the right repo?"* rather than scoping against a guess.
- ☐ If GitLab is unreachable the repo check is skipped silently and analysis proceeds.

07 Quick-impact chat

> **Where:** "Quick question" box at the top of the S3 page · any login

Try: *"how long would it take to add a vehicle-swap endorsement?"*

- ☐ Asks at most **two** clarifying questions, then must deliver a final answer.
- ☐ Final answer carries impact analysis + hour class + P-level + whether a code change is warranted.
- ☐ **Clear** resets the server-side transcript.

08 Problem-record intake — the second CR flavour

> **Run:** `demo/seed_problem_record_ticket.sh` (API must be running)

- ☐ A new ticket appears on the board tagged **origin: problem record** with its `PRB…` id.
- ☐ It runs the identical downstream flow — only the origin tag differs, which is the point: repeated incidents → problem record → CR converges on the same pipeline.

09 GitLab repo scoping — API only, no UI

```
C=$(mktemp); curl -s -c $C -X POST localhost:8000/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"name":"Ravi Kumar","passcode":"1001"}' >/dev/null

curl -s -b $C -X POST localhost:8000/api/s3/gitlab/scope-auto \
  -H 'Content-Type: application/json' \
  -d '{"tier_name":"Elite"}' | python -m json.tool
```

- ☐ The AI picks a repo from names and descriptions alone, returning confidence, reasoning and alternates.
- ☐ Below *high* confidence it returns `needs_clarification` instead of guessing.
- ☐ `repo_size` vs `files_reached_llm` shows the funnel economics — only a shortlist is ever fetched, so cost stays flat regardless of repo size.
- ☐ Nothing is ever written back to GitLab.

10 Resilience & regression

```
.venv/bin/python -m pytest tests/ -q                    # 239 pass
.venv/bin/ruff check .                                  # clean
.venv/bin/python tools/verify_s3_live.py --skip-live    # offline architecture gate
```

- ☐ Replay works with networking stubbed out entirely.
- ☐ A mid-stream provider failure falls back to replay invisibly.
- ☐ One recording replays correctly for two different tier names.
- ☐ Reset restores baseline in under 10 s.
- ☐ Core recall never drops a required file; decoy subsystems are never selected.

> **DEMO-DAY RULE**
>
> `tools/verify_s3_live.py --gate 10` runs ten real codegen + testgen + pytest cycles from baseline. A cycle passes only if the **live** path succeeded — no silent replay fallback — and the generated tests ran green. **Require ≥ 9/10, otherwise present in `LLM_MODE=replay` without hesitation.**

Known issues

#	ISSUE	IMPACT	STATUS
1	CR-2026-042 codegen recording is a net-deletion recording — strips ~6 function docstrings from `apps/policycore/core/db.py`	AMS-102 diff is dominated by unrelated deletions	**Open** — needs a live re-record
2	Subsystem screening was hardcoded to `apps/policycore/`, so the Spring target was scored against mockapp's decoy subsystems	Panel wrongly reported `legacy_java_platform/settlement` 0.49 for AMS-103	**Fixed** — target-scoped, 2 regression tests
3	Post-apply link always pointed at the mockapp portal	AMS-103 sent reviewers to the wrong app	**Fixed** — per-target app map
4	AMS-103's Jira card shows `APPLICATION: PolicyCore` though it is a ClaimsPortal CR	Cosmetic inconsistency in seeded data	Open — low priority
5	Gap gate occasionally asks about a detail the CR already states (e.g. the deductible's data type)	One wasted clarification turn	Open — prompt tuning
6	After QA hand-off the console silently re-targets another ticket, since the handed-off one leaves the developer's filtered board	Disorienting mid-demo	Open — low priority
7	ClaimsPortal consoles show no deductible column; `payableAmount` is not rendered either	No visual before/after on :8081	By design — CR forbids HTML changes

Troubleshooting

SYMPTOM	CAUSE AND FIX
UI looks logged in but every action 401s	Backend restarted (or was run with `--reload`) and cleared the in-memory session. Log out and back in; restart uvicorn without `--reload`.
:8081 or :8082 refuses to connect	Neither service is running — start with `apps/run-policy-service.sh` / `apps/run-claims-service.sh` (or `demo/run_s3_springdemo.sh` for both).
Applied a CR but the ClaimsPortal console is unchanged	Stale process — uvicorn runs without `--reload` here, so it's still serving the pre-apply code. Ctrl-C and relaunch both services.
A stage will not open	Not a bug — the grey hint under it names the missing precondition (analysis not run, change not applied, ticket not in QA, or you are not the assigned tester).
Console shows results for a ticket you just reset	`localStorage` cache. Hard-refresh the tab; the reset marker makes the console drop it.
Mutation check returns 409	Generated code drifted from the recording, so no seeded snippet matched. Re-run generate/apply, or re-record. It fails loudly rather than mutating blind.

Analysis is slow on first click	Expected — `.cache/llm` was cleared by the reset. Run `demo/warm_s3_cache.sh` to pre-warm.

All data in this demo is synthetic and the insurer is fictional. No real client data, client names, or credentials appear anywhere in this repo — see `CLAUDE.md` for the full project rules.